

Laporan Tugas 2 — REST API & HTTP Deep Dive

Mata Kuliah: Praktikum Pemrograman Backend Lanjut

Program Studi: D4 Teknik Informatika — Fakultas Vokasi — Universitas Airlangga

Pertemuan: 2 (REST API & HTTP Deep Dive)

Repositori GitHub: [https://github.com/\[username\]/\[repo\]](https://github.com/[username]/[repo])

1. Tujuan Pengerjaan

Mengangkat struct `Student` dari tugas pertemuan 1 menjadi entitas REST API utuh berbasis Go + Fiber v2, dengan perhatian pada empat hal yang membedakan API "yang berfungsi" dengan API "yang layak dipakai":

1. Pemilihan metode HTTP yang tepat (safe, idempotent).
2. Pembedaan perilaku **PUT** (ganti seluruh isi) dan **PATCH** (ubah sebagian).
3. Pemilihan status HTTP yang jujur untuk setiap situasi.
4. Rancangan query string untuk penyaringan, pengurutan, pencarian, dan paginasi.

2. Struktur Proyek

```
api-students/
├── go.mod           # module api-students + dependensi Fiber v2
├── go.sum           # checksum dependensi
├── main.go          # bootstrap Fiber + middleware + route
├── model.go         # struct entitas, DTO request, amplop respons
├── helper.go        # helper respons + parser query string
├── handler.go       # handler tiap endpoint
├── README.md        # kontrak API
└── screenshots/     # tangkapan layar pengujian
```

Keempat berkas Go tetap berada dalam `package main`, sehingga dapat saling melihat tanpa import tambahan. Pemisahan tanggung jawab yang lebih tegas (terutama antara lapisan data dan handler) baru dibahas pada pertemuan 3.

3. Penjelasan Kode Per Berkas

3.1 `model.go`

```

type Student struct {
    ID          int    `json:"id"`
    NIM         string  `json:"nim"`
    Name        string  `json:"name"`
    Grade       float64 `json:"grade"`
    IsActive    bool   `json:"is_active"`
    CreatedAt   time.Time `json:"created_at"`
}

```

Struct `Student` adalah kelanjutan dari struct pada tugas pertemuan 1 (`ID`, `Name`, `Grade`, `IsActive`) dengan penambahan field `NIM` sebagai **penanda unik** mahasiswa.

Mengapa DTO request dipisah untuk POST, PUT, dan PATCH?

```

type CreateStudentRequest struct {
    NIM, Name string
    Grade     float64
    IsActive  bool
}

type ReplaceStudentRequest struct {
    NIM, Name string
    Grade     float64
    IsActive  bool
}

type PatchStudentRequest struct {
    NIM      *string `json:",omitempty"`
    Name     *string `json:",omitempty"`
    Grade    *float64 `json:",omitempty"`
    IsActive *bool   `json:",omitempty"`
}

```

- **POST** dan **PUT** menggunakan tipe field biasa. PUT mewajibkan semua field karena PUT berarti *mengganti seluruh isi*; field yang tidak dikirim akan dianggap kosong (zero value).
- **PATCH** menggunakan tipe **pointer** untuk semua field. Tujuannya supaya server bisa membedakan "tidak dikirim" (`nil`) dari "dikirim bernilai default" (misalnya `false` untuk `IsActive`). Tanpa pointer, `PATCH {"is_active": false}` dan PATCH tanpa body akan terlihat sama oleh server.

Mengapa satu struct untuk semua tidak dipakai?

- Klien bisa mengirim `{"id": 999}` untuk menimpa ID server.
- Klien bisa mengirim `{"created_at": "1970-01-01"}` untuk memalsukan waktu.
- Memakai struct yang sama akan mewajibkan klien mengirim field yang seharusnya tidak boleh diubah.

```

type WebResponse struct {
    Success bool   `json:"success"`
    Message string `json:"message"`
    Data    any    `json:"data,omitempty"`
    Meta    *Meta  `json:"meta,omitempty"`
    Errors  any    `json:"errors,omitempty"`
}

type Meta struct {
    Page, Limit, Total, TotalPages int
}

```

`WebResponse` adalah **amplop tunggal** untuk seluruh respons (berhasil maupun gagal). Konsistensi bentuk membuat klien tidak perlu menebak struktur per endpoint. `Meta` membawa informasi paginasi agar klien dapat menggambarkan navigasi halaman tanpa menebak.

3.2 `helper.go`

```

func ok(c *fiber.Ctx, message string, data any) error {
    return c.Status(fiber.StatusOK).JSON(WebResponse{
        Success: true, Message: message, Data: data,
    })
}

func created(c *fiber.Ctx, message string, data any, location string) error {
    c.Set("Location", location)
    return c.Status(fiber.StatusCreated).JSON(WebResponse{
        Success: true, Message: message, Data: data,
    })
}

func noContent(c *fiber.Ctx) error {
    return c.SendStatus(fiber.StatusNoContent)
}

func failValidation(c *fiber.Ctx, errs map[string]string) error {
    return c.Status(fiber.StatusUnprocessableEntity).JSON(WebResponse{
        Success: false, Message: "validasi gagal", Errors: errs,
    })
}

```

Empat helper ini memastikan **setiap** endpoint memakai status dan bentuk respons yang konsisten. `created()` menyetel header `Location` sesuai RFC 9110 agar klien tahu alamat resource baru. `failValidation()` mengirim 422 dengan rincian kesalahan **per field**, sehingga aplikasi klien dapat menandai kolom yang salah pada formulir.

Parser query string dengan whitelist:

```

var allowedSort = map[string]bool{
    "id": true, "nim": true, "name": true,
    "grade": true, "created_at": true,
}

func parseListQuery(c *fiber.Ctx) ListQuery {
    q := ListQuery{
        Page:    c.QueryInt("page", 1),
        Limit:    c.QueryInt("limit", 10),
        Search:   strings.TrimSpace(c.Query("search")),
        Sort:     c.Query("sort", "id"),
        Order:    strings.ToLower(c.Query("order", "asc")),
    }
    if q.Page < 1 { q.Page = 1 }
    if q.Limit < 1 { q.Limit = 10 }
    if q.Limit > 100 { q.Limit = 100 } // batas atas wajib
    if !allowedSort[q.Sort] { q.Sort = "id" } // whitelist
    ...
}

```

- **Batas atas** `limit=100` mencegah klien mengirim `?limit=99999999` yang bisa menghabiskan memori server. Ini bukan skenario hipotetis.
- **Daftar putih untuk** `sort` mencegah input apapun dari klien ditempelkan ke query. Walau data saat ini masih di memori, kebiasaan ini dibentuk sekarang karena akan diterjemahkan langsung ke SQL ketika basis data masuk pada pertemuan 3.

Middleware `requireJSON` :

```

var metodeBerbody = map[string]bool{
    fiber.MethodPost: true, fiber.MethodPut: true, fiber.MethodPatch: true,
}

func requireJSON(c *fiber.Ctx) error {
    if metodeBerbody[c.Method()] {
        ct := c.Get("Content-Type")
        if !strings.HasPrefix(ct, fiber.MIMEApplicationJSON) {
            return fail(c, fiber.StatusUnsupportedMediaType,
                "Content-Type harus application/json")
        }
    }
    return c.Next()
}

```

`requireJSON` dipasang **khusus pada grup** `/users`, bukan global. Kalau dipasang global, endpoint yang menerima unggahan berkas (`multipart/form-data`) akan ikut tertolak.

3.3 `handler.go`

Daftar + paginasi + sortir + saringan:

```

sort.SliceStable(hasil, func(i, j int) bool {
    var lebihKecil bool
    switch q.Sort {
    case "nim":    lebihKecil = hasil[i].NIM < hasil[j].NIM
    case "name":  lebihKecil = hasil[i].Name < hasil[j].Name
    case "grade": lebihKecil = hasil[i].Grade < hasil[j].Grade
    default:      lebihKecil = hasil[i].ID < hasil[j].ID
    }
    if q.Order == "desc" { return !lebihKecil }
    return lebihKecil
})

total := len(hasil)
totalPages := (total + q.Limit - 1) / q.Limit // ceiling division
mulai := (q.Page - 1) * q.Limit
if mulai > total { mulai = total }
akhir := mulai + q.Limit
if akhir > total { akhir = total }

return okList(c, "...", hasil[mulai:akhir], &Meta{
    Page: q.Page, Limit: q.Limit, Total: total, TotalPages: totalPages,
})

```

Alur: **saring** → **urutkan** → **potong sesuai halaman**. Pemotongan dilakukan setelah penyaringan dan pengurutan agar **meta.total** merepresentasikan jumlah data **setelah** filter, bukan jumlah total di memori.

POST dengan validasi dan 409:

```

if len(errs) == 0 && findStudentByNIM(req.NIM, 0) {
    return fail(c, fiber.StatusConflict, "NIM sudah dipakai")
}

```

Status 409 dipakai untuk **konflik data** (NIM duplikat), bukan 422. Alasannya, kesalahan 422 berarti "isi tidak lolos validasi"; NIM duplikat bukan salah isi melainkan bentrok dengan data yang sudah ada.

PUT (ganti seluruh):

```

users[i].NIM    = req.NIM
users[i].Name   = req.Name
users[i].Grade  = req.Grade
users[i].IsActive = req.IsActive

```

PATCH (ubah sebagian):

```

if req.Grade != nil {
    if *req.Grade < 0 || *req.Grade > 100 {
        return failValidation(c, map[string]string{"grade": "..."})
    }
    students[i].Grade = *req.Grade
}
if req.IsActive != nil {
    students[i].IsActive = *req.IsActive
}

```

Pointer `*req.Grade` hanya dipakai di dalam blok `if req.Grade != nil`, sehingga field yang tidak dikirim benar-benar **tidak disentuh**.

DELETE:

```
students = append(students[:i], students[i+1:]...)
return noContent(c) // 204 tanpa body
```

Menggunakan irisan Go untuk menghapus elemen slice secara efisien.

3.4 `main.go`

```
app.Use(requestid.New())
app.Use(logger.New(logger.Config{
    Format: "[%{time}] ${locals:requestid} ${method} ${path} ${status} ${latency}\n",
}))
app.Use(cors.New())

s := api.Group("/students", requireJSON)
s.Get("/", listStudents)
s.Get("/:id", getStudent)
s.Post("/", createStudent)
s.Put("/:id", replaceUser)
s.Patch("/:id", patchUser)
s.Delete("/:id", deleteUser)
```

- Urutan middleware = urutan eksekusi. `requestid` dipasang **pertama** agar logger di belakangnya bisa merujuk `${locals:requestid}`.
- `ErrorHandler` kustom menerjemahkan `*fiber.Error` (error bawaan Fiber) menjadi amplop `WebResponse`, sehingga 404 dari router Fiber tidak bocor dalam bentuk polos.

4. Bukti Pengujian Tiap Status HTTP

Bagian ini memuat tangkapan layar pengujian. Setiap tangkapan layar diambil dengan `curl -i` agar **baris status HTTP** terlihat jelas, bukan hanya isi body.

Tempatkan tangkapan layar Anda di folder `screenshots/` sebelum export PDF. Bagian ini dapat diganti dengan gambar hasil tangkapan layar Anda.

4.1 Status 200 — Pengambilan Berhasil

- `[screenshots/01-get-health.png]` — `GET /api/v1/health`
- `[screenshots/02-get-list.png]` — `GET /api/v1/students?page=1&limit=2`
- `[screenshots/03-get-one.png]` — `GET /api/v1/students/1`

4.2 Status 201 — Pembuatan Berhasil (disertai `Location`)

- `[screenshots/04-post-create.png]` — `POST /api/v1/students`

- Perhatikan header `Location: /api/v1/students/1` dan `X-Request-Id`.

4.3 Status 200 — PUT Mengganti Seluruh Isi

- `[screenshots/05-before.png]` — keadaan sebelum PUT.
- `[screenshots/06-put.png]` — permintaan PUT dengan semua field.
- `[screenshots/07-after-put.png]` — keadaan sesudah PUT. **Seluruh** field berubah, termasuk `is_active` dari `true` ke `false`.

4.4 Status 200 — PATCH Mengubah Sebagian

- `[screenshots/08-patch.png]` — permintaan PATCH hanya `{"grade":95.5}`.
- `[screenshots/09-after-patch.png]` — keadaan sesudah PATCH. Hanya `grade` yang berubah; `nim`, `name`, dan `is_active` **tetap**.

4.5 Status 204 — Penghapusan Berhasil (tanpa body)

- `[screenshots/10-delete.png]` — `DELETE /api/v1/students/2`. Respons hanya berisi header, tanpa body.

4.6 Status 400 — Permintaan Tidak Sah

- `[screenshots/11-bad-json.png]` — body bukan JSON.
- `[screenshots/12-bad-id.png]` — `:id` bukan angka (`/students/abc`).

4.7 Status 404 — Sumber Daya Tidak Ditemukan

- `[screenshots/13-not-found.png]` — `GET /api/v1/students/999`.

4.8 Status 409 — Konflik Data

- `[screenshots/14-conflict-nim.png]` — POST dengan NIM yang sudah dipakai mahasiswa lain.

4.9 Status 415 — Content-Type Tidak Didukung

- `[screenshots/15-wrong-ct.png]` — POST tanpa header Content-Type.

4.10 Status 422 — Validasi Isi Gagal (dengan rincian per field)

- `[screenshots/16-validation.png]` — POST dengan `grade=150`. Respons berisi `errors:`
`{ "grade": "harus bernilai antara 0 dan 100" }`.

5. Kontrak API (Ringkas)

Metode	Endpoint	Status
GET	/api/v1/students	200
GET	/api/v1/students/:id	200, 400, 404
POST	/api/v1/students	201 (+ Location), 400, 409, 415, 422
PUT	/api/v1/students/:id	200, 400, 404, 409, 415, 422
PATCH	/api/v1/students/:id	200, 400, 404, 409, 415, 422
DELETE	/api/v1/students/:id	204, 400, 404

Query string yang didukung:

Parameter	Nilai bawaan	Keterangan
<code>page</code>	<code>1</code>	Halaman ke berapa
<code>limit</code>	<code>10</code> (maks 100)	Baris per halaman (batas atas = 100)
<code>search</code>	(kosong)	Pencarian pada <code>name</code> , case-insensitive
<code>sort</code>	<code>id</code>	Whitelist: <code>id</code> , <code>nim</code> , <code>name</code> , <code>grade</code> , <code>created_at</code>
<code>order</code>	<code>asc</code>	<code>asc</code> atau <code>desc</code>
<code>is_active</code>	(kosong)	Filter <code>true</code> / <code>false</code>
<code>grade_min</code>	(kosong)	<code>grade >= grade_min</code>
<code>grade_max</code>	(kosong)	<code>grade <= grade_max</code>

6. Refleksi: PUT vs PATCH

Pengujian di bagian 4.3 dan 4.4 membuktikan bahwa perbedaan PUT dan PATCH bukan sekadar nama metode:

- **PUT** mengirim `{"nim":..., "name":..., "grade":90, "is_active":false}` dan mengubah **seluruh** field record. Bila `is_active` tidak dikirim, nilainya akan menjadi `false` (zero value) meskipun sebelumnya `true`.
- **PATCH** mengirim `{"grade":95.5}` dan hanya mengubah `grade`. Field `nim`, `name`, dan `is_active` **tidak disentuh** karena pointer-nya `nil`.

Inilah kegunaan pointer yang diajarkan di pertemuan 1: tanpa `*bool`, tidak ada cara bagi server untuk membedakan "tidak dikirim" dari "dikirim bernilai `false`".

7. Sumber Bantuan

- Modul Praktikum Pertemuan 2 — REST API & HTTP Deep Dive (pengajar).

- Dokumentasi resmi [Fiber v2](#) untuk middleware, status code, dan `BodyParser` .
- [MDN Web Docs — HTTP](#) untuk daftar status dan header.
- Alat bantu AI digunakan untuk menyusun pola pemisahan berkas sesuai modul dan memeriksa konsistensi pesan validasi. Logika bisnis, validasi, struktur endpoint, dan seluruh pengujian ditulis sendiri.

Halaman akhir — laporan dikumpulkan sebagai `Tugas2_NIM.pdf` .